

Code Assessment of the CCIPDVNAdapter Smart Contracts

PUBLIC

Date [June 23, 2026](#)

Produced for



By



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	7
5	System Considerations	10
6	Terminology	11
7	Findings	12
8	Limitations and use of report	15

1 Executive Summary

Dear all,

Thank you for trusting us to help Sky with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed version of CCIPDVNAdapter according to [Scope](#) to support you in forming an opinion on their security risks.

CCIPDVNAdapter is a Chainlink CCIP adapter for the LayerZero v2 messaging protocol, developed by LayerZero. Sky intends to use it unmodified as the Chainlink CCIP verification path of a cross-chain governance bridge.

The most critical subjects covered in our audit are the correctness of the verification relay, the authentication of inbound cross-chain messages, and access control. Security regarding all the aforementioned subjects is high.

The general subjects covered are the robustness of the destination configuration, the dependency on Chainlink CCIP, and code complexity. Security regarding all the aforementioned subjects is high.

The adapter places extensive power in its privileged roles and delegates message finality and transport to Chainlink CCIP, which are trusted accordingly; see the [Trust Model](#).

In summary, we find that the codebase provides a high level of security, provided it is configured correctly.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code of the `CCIPDVNAdapter` and `CCIPDVNAdapterFeeLib` contracts in the LayerZero v2 repository, based on the provided documentation. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	15 June 2026	9c741e7f9790639537b1710a203bcdfd73b0b9ac	Initial Review

No fixed compiler version is configured. The contracts specify a floating pragma `^0.8.20` and therefore compile with any 0.8.x version from 0.8.20 onwards.

The following files are in scope:

```
packages/layerzero-v2/evm/messagelib/contracts/uln/dvn/adapters/CCIP/  
├── CCIPDVNAdapter.sol  
└── CCIPDVNAdapterFeeLib.sol
```

The base contracts that `CCIPDVNAdapter` inherits, namely `DVNAdapterBase` and `Worker`, together with the `DVNAdapterMessageCodec` library used through them, are in scope insofar as `CCIPDVNAdapter` relies on them.

2.1.1 Excluded from scope

All other contracts in the LayerZero v2 repository are excluded from scope. This includes in particular the remaining LayerZero protocol contracts (the endpoint, the message libraries, the executor, and the receive library) and the Chainlink CCIP contracts. These components, together with their off-chain counterparts such as the Chainlink CCIP network, are treated as trusted and are not reviewed.

3 System Overview

This system overview describes the initially received version (`Version 1`) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

The `CCIPDVNAdapter` is a Chainlink CCIP adapter for the LayerZero v2 messaging protocol by LayerZero. In LayerZero v2, an application selects a set of Decentralized Verifier Networks (DVNs) that must each independently attest to a cross-chain message before the destination endpoint delivers it. The `CCIPDVNAdapter` is one such DVN that, instead of operating a dedicated off-chain network, relays the attestation from the source chain to the destination chain over Chainlink CCIP.

3.1 CCIPDVNAdapter

`CCIPDVNAdapter` is a single contract deployed on each supported chain that plays two roles. On the source chain it implements the LayerZero DVN interface (`ILayerZeroDVN`): when a message is sent, the source message library assigns it a verification job, and the adapter forwards the attestation as a CCIP message. On the destination chain it implements the CCIP receiver interface (`IAny2EVMMessageReceiver`): the CCIP Router delivers that message to the adapter, which authenticates it and records the attestation by calling `verify()` on the destination receive library. Each deployment is configured with the counterpart adapter on the other chain as its trusted peer, and accepts an inbound attestation only when it arrives through the CCIP Router from that peer.

When `assignJob()` is called, the adapter encodes the message identity (the destination receive library, the packet header, and the payload hash) with `DVNAdapterMessageCodec`, wraps it into a CCIP `EVM2AnyMessage` addressed to the configured peer with a per-destination gas limit, and queries the fee library for the CCIP fee plus the configured premium. It then ensures it holds enough native balance to cover the CCIP fee, pulling its accrued worker fees from the calling message library if necessary, and calls `ccipSend()` on the Router paying that fee in the native gas token.

On receipt, the adapter accepts a call only from the CCIP Router. It resolves the source configuration from the CCIP chain selector, requires that the CCIP message sender equals the configured trusted peer, decodes the payload, and requires that the encoded source endpoint id (normalized modulo 30000) matches the configured value before calling `verify()` on the decoded receive library. The attestation is always recorded with the maximum confirmation count (`type(uint64).max`): the adapter delegates finality entirely to CCIP and does not honor the confirmation count requested by the application.

Configuration is keyed by endpoint id normalized modulo 30000. LayerZero identifies each chain by a numeric endpoint id, and the Endpoint V2 id equals the Endpoint V1 id plus 30000; reducing modulo 30000 maps both to the same key, so a single adapter with one configuration entry per chain serves verification jobs from the message libraries of both Endpoint V1 and Endpoint V2. For each destination, the CCIP chain selector and the trusted peer are written once and are immutable afterwards. On the source chain, the receive library to verify against on the destination is read from an inherited mapping keyed by the source message library and the destination endpoint, encoded into the message, and only decoded and called on the destination. From its base contracts the adapter also inherits worker fee accounting and withdrawal, role based access control, and an allow and deny list over which applications may use it: `assignJob()` and `getFee()` call `onlyAc1()` on the application (`_param.sender`), which rejects a denylisted application and, when an allowlist is configured, requires the application to be on it; with the default empty allowlist any application is allowed. Two further inherited features are not used: the pausing flag (no function applies the `whenNotPaused` modifier, so pausing has no effect) and the price feed (never read, since the fee is quoted by the CCIP Router).

The contract provides the following functions:

- `assignJob()` : called by a configured LayerZero message library; encodes the attestation, sends it over CCIP to the peer, and pays the CCIP fee.
- `getFee()` : view function returning the fee charged for a given job (the CCIP fee plus the configured premium).
- `ccipReceive()` : callable only by the CCIP Router; authenticates the source peer and records the verification on the destination receive library.
- `setDstConfig()` : configures, per destination endpoint id, the CCIP chain selector, trusted peer, destination gas limit, and fee multiplier; the chain selector and peer are set only on first configuration.
- `supportsInterface()` : advertises support for the CCIP receiver interface.

Inherited from `DVNAdapterBase` and `Worker`, the adapter additionally exposes `setReceiveLibs()` to configure the destination receive library per source message library and destination endpoint, `receive()` to be funded with native value for CCIP fees, and the worker administration functions (`setWorkerFeeLib()`, `setDefaultMultiplierBps()`, `setPriceFeed()`, `setSupportedOptionTypes()`, `withdrawFee()`, `withdrawToken()`, `setPaused()`).

3.2 CCIPDVNAdapterFeeLib

`CCIPDVNAdapterFeeLib` is the fee library the adapter calls. It is an upgradeable contract. Both `getFeeOnSend()` and `getFee()` quote the underlying CCIP cost through `router.getFee()` and apply the worker premium ($\text{fee} * \text{multiplierBps} / 10000$, falling back to the adapter's default multiplier when the per destination value is zero); both revert if the destination gas is zero or if any message options are supplied. `getFeeOnSend()` additionally returns the raw CCIP fee that the adapter forwards to the Router.

The contract provides the following functions:

- `getFeeOnSend()` : called by the adapter during `assignJob()`; returns the raw CCIP fee and the total with the premium applied.
- `getFee()` : view function returning the total with the premium applied for a quote.
- `setDstConfig()` : callable only by the owner; stores a per destination `floorMarginUSD`.
- `initialize()` : proxy initializer that sets the owner.

4 Trust Model

This section enumerates the privileged roles and trusted external components of the CCIP DVN adapter.

Note: the adapter contributes a single attestation within an application's DVN set. Unless an OApp configures this adapter as its only required verifier, none of the roles below can forge a cross-chain message. The worst achievable outcomes are loss of the adapter's own funds and denial of service (the adapter fails to attest, so messages requiring it stall).

4.1 Roles

4.1.1 DEFAULT_ADMIN_ROLE

- **Capabilities:** Admin of every role, so it can grant and revoke all roles, including itself. Directly gates `setReceiveLibs()` and `setPaused()`.
- **Trust level:** Fully trusted.
- **Worst case:** Effectively owns the adapter. It can grant itself `ADMIN_ROLE` and acquire every capability below, rewrite `receiveLibs[sendLib][dstEid]` to redirect or disable the destination verification target, and (via the self granted `ADMIN_ROLE`) set a malicious peer at initial configuration, which is the only path to verification forgery. It can also denylist any application.

4.1.2 ADMIN_ROLE

- **Capabilities:** The operational surface: `setDstConfig()`, `setWorkerFeeLib()`, `setDefaultMultiplierBps()`, `setPriceFeed()`, `setSupportedOptionTypes()`, `withdrawFee()`, `withdrawToken()`. It cannot grant or revoke roles.
- **Trust level:** Fully trusted.
- **Worst case (funds):** `withdrawToken(address(0), to, amount)` sends the adapter's native balance (the buffer that prepays CCIP fees) to any address, and `withdrawFee()` pulls accrued worker fees out of any `SendLib` granted `MESSAGE_LIB_ROLE`.
- **Worst case (fee hot path):** `setWorkerFeeLib()` repoints the fee library used by `assignJob()` / `getFee()`. A malicious fee lib can overcharge applications, revert to cause a denial of service, or return an inflated `ccipFee`.
- **Worst case (configuration):** `setDstConfig()` writes `chainSelector` and `peer` only once (while `dstConfig[eid].chainSelector == 0`). A malicious `peer` at initial configuration lets an address the attacker controls pass `_assertPeer()` and drive `verify()` with forged data. After the first write the peer is immutable.

4.1.3 CCIPDVNAdapterFeeLib owner and proxy admin

- **Capabilities:** The fee library is an upgradeable contract. Its proxy admin can replace the implementation. Its `owner` can transfer ownership and call `setDstConfig()`, which stores a per destination `floorMarginUSD`; that value is never read by the implementation, so setting it has no effect. The adapter calls this library on every `assignJob()` and `getFee()`.
- **Trust level:** Fully trusted.
- **Worst case:** Control of the implementation (the proxy admin, or `ADMIN_ROLE` repointing `workerFeeLib`) controls fee computation in the adapter's hot path: it can overcharge, revert to cause a denial of service, or inflate `ccipFee` to drain the adapter's prepaid balance. The `owner` role on its own has no effect on fees.

4.1.4 ALLOWLIST / DENYLIST

- **Capabilities:** `assignJob()` and `getFee()` gate the application `_param.sender` through the custom `onlyAcl()` modifier rather than a role check. `hasAcl()` denies a denylisted sender, otherwise allows it when the allowlist is empty or lists it. By default `allowlistSize == 0`, so the adapter is open to all applications.
- **Trust level:** Partially trusted.
- **Worst case:** The grief vector is the granting power held by `DEFAULT_ADMIN_ROLE`.

4.1.5 SendLib (message library)

- **Capabilities:** The `msg.sender` of `assignJob()` / `getFee()`. The effective gate is `_getAndAssertReceiveLib(msg.sender, dstEid)`, which reverts unless `receiveLibs[msg.sender][dstEid]` is configured (set only by `DEFAULT_ADMIN_ROLE`). It supplies the `packetHeader` / `payloadHash` that the adapter relays as its attestation, triggers the `ccipSend` that spends the adapter's prepaid native, and exposes its accrued fees to `withdrawFee()`.
- **Trust level:** Fully trusted.
- **Worst case:** A malicious or compromised configured SendLib can drive the adapter to attest to arbitrary `packetHeader` / `payloadHash` on the destination and drain the prepaid native balance through repeated sends. It must first be authorized in `receiveLibs` by `DEFAULT_ADMIN_ROLE`.

4.1.6 Unprivileged actors

- **Application / OApp:** identified by `_param.sender` and subject to the ACL. Untrusted; it cannot exceed the ACL and pays the fee including the premium.
- **End users / anyone:** `receive()` accepts native value (for refunds), the view getters are public, `ccipReceive()` reverts for any caller other than the CCIP Router, and `renounceRole()` reverts (role renouncing is disabled). Untrusted, with no privileged capability.

4.2 Trusted external components

- **Chainlink CCIP:** On send, the adapter calls `router.getFee()` and `router.ccipSend()`; on receive, the CCIP Router is the sole authorized caller of `ccipReceive()`. CCIP's honesty and liveness are this DVN's verification security and availability.
- **The sibling CCIPDVNAdapter (trusted peer):** authenticated in `ccipReceive()` via `_assertPeer()` against the set once `srcConfig[selector].peer`. Anything that peer sends is accepted as a valid attestation source.
- **ReceiveULN (IReceiveUln):** `_decodeAndVerify()` calls `verify(packetHeader, payloadHash, MAX_CONFIRMATIONS)` on the receive library address decoded from the inbound message.
- **SendLib (ISendLib):** the source of fees pulled in `_withdrawFeeFromSendLib()` / `withdrawFee()`.

4.3 Assumptions for external components

- CCIP's DON correctly and exclusively delivers messages that genuinely originated from the configured peer, and finalizes them appropriately. The adapter delegates finality entirely to CCIP: it always verifies at `MAX_CONFIRMATIONS` and does not honor the application's requested block confirmations (`AssignJobParam.confirmations` is forwarded to the fee lib but otherwise ignored; `verify()` uses `type(uint64).max`).
- `router.getFee()` returns a fee valid for the same transaction's `ccipSend()`; the adapter quotes and pays within a single call.

- The trusted peer is configured correctly at the first `setDstConfig()` (immutable thereafter).
- The configured `receiveLibs` targets are honest LayerZero/CCIP system contracts.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Unused Functionality

System Consideration	Version 1
----------------------	-----------

`CCIPDVNAdapter` inherits several features from `Worker`, the shared base used by other LayerZero workers, that are configurable but have no effect here:

- Pausing: `Worker` exposes `setPaused()` and OpenZeppelin `Pausable`, but no function in `CCIPDVNAdapter`, `DVNAdapterBase`, or `Worker` applies the `whenNotPaused` modifier. Pausing therefore has no effect on `assignJob()`, `getFee()`, or `ccipReceive()`.
- Price feed: `priceFeed` is settable through `setPriceFeed()` but is never read. The adapter obtains its fee from the CCIP Router, not from a price feed.
- Supported option types: `setSupportedOptionTypes()` and `getSupportedOptionTypes()` maintain a per endpoint list of option types that the adapter never uses.

`CCIPDVNAdapterFeeLib` stores a configured value that its fee functions never read:

- Floor margin: `CCIPDVNAdapterFeeLib.setDstConfig()` stores a per destination `floorMarginUSD`, and the interface documents it as "uses priceFeed `PRICE_RATIO_DENOMINATOR`". However, `getFeeOnSend()` and `getFee()` compute the fee solely as the CCIP fee from `router.getFee()` scaled by the multiplier `(fee * multiplierBps / BPS_DENOMINATOR)`; `floorMarginUSD` is never read, and the contract holds no price feed. The documented USD floor is therefore not applied: the configured value has no effect, and an operator setting it obtains no minimum fee in USD terms.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

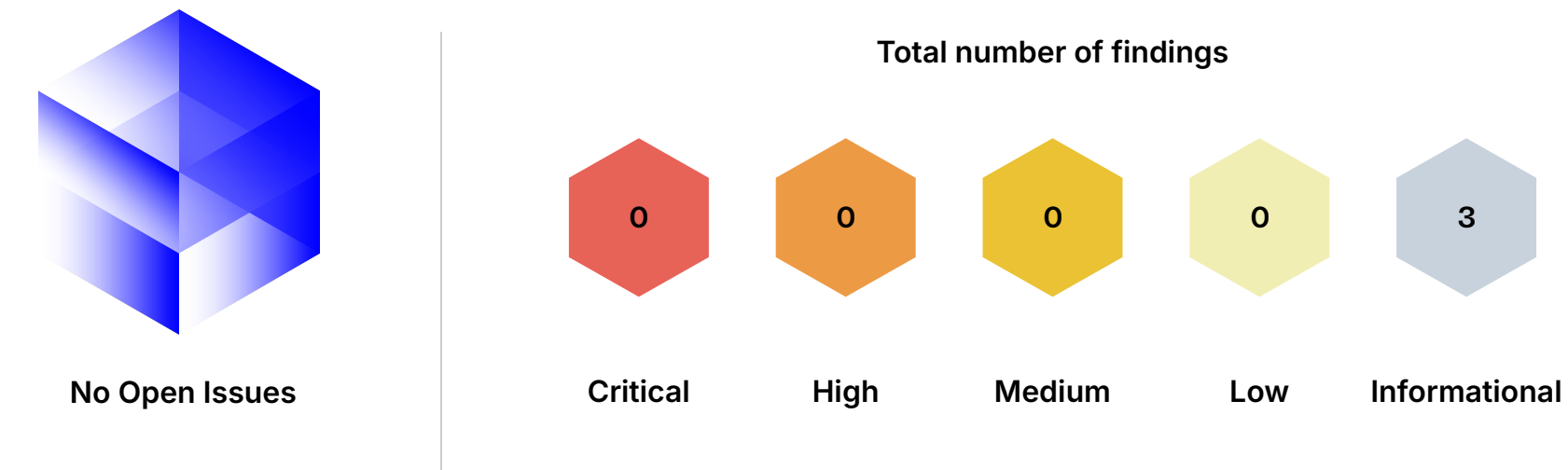
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Info	#001	Adapter Cannot Send over CCIP Lanes That Enforce Out of Order Execution	Acknowledged	⊖
Info	#002	setDstConfig Lacks Input Validation and Writes Irreversible Configuration	Acknowledged	⊖
Info	#003	withdrawToken Emits No Event	Acknowledged	⊖

7.2 Descriptions

#001 Adapter Cannot Send over CCIP Lanes That Enforce Out of Order Execution

Informational	Version 1	Acknowledged 
---------------	-----------	--

`_createCCIPMessage()` sets the `extraArgs` field of the CCIP `EVM2AnyMessage` to `Client._argsToBytes(Client.EVMExtraArgsV1({ gasLimit: _gas, strict: false })))`.

The pinned `@chainlink/contracts-ccip` version is `0.7.6`, whose `Client` library defines only `EVMExtraArgsV1 { uint256 gasLimit; bool strict }` and no `V2` struct. Current CCIP has removed `strict` (its `EVMExtraArgsV1` is `{ uint256 gasLimit }`, and the `OnRamp` ignores any `strict` bytes a message carries) and added `EVMExtraArgsV2 { uint256 gasLimit; bool allowOutOfOrderExecution }`, also called `GenericExtraArgsV2` in some CCIP references.

On a legacy `v1.5.x` `EVM2EVMOnRamp`, `_fromBytes()` decodes the `V1` tag to `allowOutOfOrderExecution = false`. When that lane enforces out of order execution the `OnRamp` reverts with `ExtraArgOutOfOrderExecutionMustBeTrue` in `_validateMessage()`, which runs both in the send path (`ccipSend()` calls `forwardFromRouter()`) and in `EVM2EVMOnRamp.getFee()` (which `Router.getFee()` delegates to). The adapter hits the quote first: it reaches `Router.getFee()` through the fee library in both `assignJob()` (via `getFeeOnSend()`, before `ccipSend()` is reached) and the standalone `getFee()` quote, so it reverts already at fee quoting and can neither quote a fee nor deliver an attestation. Because it emits `EVMExtraArgsV1` (and `0.7.6` provides no `EVMExtraArgsV2`) this cannot be changed without upgrading the CCIP dependency. Destination lanes that enforce out of order execution are unusable.

This applies only to legacy `EVM2EVMOnRamp` (`v1.5.x`) lanes. On a `v1.6.0` lane the `OnRamp` delegates `extraArgs` parsing to its `FeeQuoter`; the deployed `FeeQuoter 2.0.0` forces `allowOutOfOrderExecution = true` for every tag (`V1` included) and has no out of order enforcement, so it accepts the adapter's `V1` message. The issue is therefore limited to legacy `v1.5.x` lanes that enable `enforceOutOfOrder == true`, a per lane CCIP setting that is independent of the `OnRamp` version and can be toggled by CCIP. As of the time of writing, from Ethereum mainnet the Linea, Scroll, Polygon zkEVM, Zircuit, Abstract, Lens, and Merlin lanes run `EVM2EVMOnRamp 1.5.0` with `enforceOutOfOrder` enabled and reject the adapter's message.

Acknowledged

Sky is aware but decided not to change LayerZero's code.

#002 setDstConfig Lacks Input Validation and Writes Irreversible Configuration

Informational	Version 1	Acknowledged 
---------------	-----------	--

`setDstConfig()` performs no validation on `param.chainSelector`, `param.peer`, `param.gas`, or `param.multiplierBps`. `gas` and `multiplierBps` are rewritten on every call and stay updatable, but `chainSelector` and `peer` (and the `srcConfig` entry derived from them) are written only on the first initialization for an `eid` and cannot be changed afterwards.

The initialization branch is gated on `dstConfig[eid].chainSelector == 0`, keyed by `eid`, but it also writes `srcConfig[param.chainSelector]`, keyed by `chainSelector`, with no guard that the `chainSelector` is unused:

- **Reassigning a chainSelector.** Configuring a fresh `eid B` with a `chainSelector X` that is already mapped to `eid A` enters the branch again (because `dstConfig[B].chainSelector == 0`) and overwrites `srcConfig[X] = {B, peerB}`. Inbound CCIP messages from chain `X` are then checked against `peerB` and `eid B`, so messages from the original peer fail `_assertPeer()` and are rejected: an inbound denial of

service on eid `A` . After `% 30000` normalization a chainSelector maps to a single normalized eid, so this is an operator error rather than a legitimate shared selector.

- **chainSelector == 0 not rejected.** Initializing with `chainSelector = 0` leaves `dstConfig[eid].chainSelector == 0` , so the gate never closes: the entry (including its `peer`) stays mutable despite the intended single initialization, and a stray `srcConfig[0]` entry is written. The `srcConfig[0]` entry is harmless because `ccipReceive()` never sees a zero source chain selector, but the unclosed gate defeats the immutability of `peer` .
- **Empty peer .** Initializing with `peer = ""` and a nonzero chainSelector permanently locks the empty peer: outbound `ccipSend()` fails for lack of a valid receiver, and inbound `_assertPeer()` rejects every real sender (`keccak256(sender) != keccak256("")`). The eid is bricked.

Acknowledged

Sky is aware but decided not to change LayerZero's code.

#003 withdrawToken Emits No Event

Informational	Version 1	Acknowledged 
---------------	-----------	--

`Worker.withdrawToken` lets `ADMIN_ROLE` transfer any ERC-20 tokens or ETH from the worker to an arbitrary address without emitting an event, so off-chain indexers or treasury monitoring that track emitted events would not observe the withdrawal. In contrast, the sibling `withdrawFee` emits `Withdraw(_lib, _to, _amount)` for the equivalent fee withdrawal.

Acknowledged

Sky is aware but decided not to change LayerZero's code.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.